

ANGULAR 101

Applications Medicales & Sante Numerique

Construire un Systeme de Gestion de Patients

De zero a une application Angular medicale prete pour la production

Duree : 4 Heures | Niveau : Debutant a Intermediaire

Prerequis : HTML, CSS & JavaScript de base | Notions de TypeScript souhaitees

*AVERTISSEMENT : Les donnees patients utilisees dans cet atelier sont fictives et ne constituent pas des donnees medicales reelles.
Pour CI-GBM2-TTIS*

Programme & Calendrier de l'Atelier

Cet atelier intensif de 4 heures couvre tous les concepts fondamentaux d'Angular a travers des exemples concrets tires du domaine medical. L'application fil rouge est un Systeme de Gestion de Patients (SGP) incluant fiches patients, ordonnances, rendez-vous et tableau de bord clinique.

0:00 - 0:30	Module 1 : Vue d'ensemble Angular & Architecture (30 min)
0:30 - 0:55	Module 2 : Installation & Premier Projet Medical (25 min)
0:55 - 1:35	Module 3 : Composants Medicaux Approfondis (40 min)
1:35 - 2:05	Module 4 : Templates & Liaison de Donnees (30 min)
2:05 - 2:35	Module 5 : Directives Integrees & Personnalisees (30 min)
2:35 - 3:00	Module 6 : Services & Injection de Dependances (25 min)
3:00 - 3:25	Module 7 : Routage Angular Medical (25 min)
3:25 - 3:45	Module 8 : Formulaires Reactifs Medicaux (20 min)
3:45 - 4:00	Module 9 : HTTP Client, FHIR & Securite (15 min)

Objectifs Pedagogiques - A l'issue de cet atelier vous serez capable de :

- Expliquer l'architecture Angular : modules, composants, services et injection de dependances
- Creer des composants medicaux avec @Input/@Output (fiche patient, carte ordonnance)
- Utiliser les 4 types de liaison de donnees et les directives integrees
- Construire des formulaires reactifs valides pour la saisie de donnees patients
- Naviguer entre les vues (liste patients, dossier, consultation) avec Angular Router
- Consommer des API REST conformes FHIR avec HttpClient et les Observables
- Appliquer les bonnes pratiques de securite pour les applications medicales (HDS, RGPD)

Module 1 : Vue d'ensemble Angular & Architecture

Duree : 30 minutes | Format : Cours Magistral + Discussion

1.1 Pourquoi Angular pour les Applications Medicales ?

Angular est un framework complet developpe et maintenu par Google. Dans le domaine medical, sa solidite architecturale, son support natif de TypeScript (typage fort, interfaces), et son ecosysteme mature en font un choix privilegie pour les applications critiques comme les Dossiers Patients Informatises (DPI), les systemes de telemedecine ou les tableaux de bord cliniques.

Critere	Angular	React	Vue
Type	Framework complet	Bibliotheque UI	Framework progressif
Langage	TypeScript natif	JS / TSX	JS / TS
Architecture	Imposee (fiable)	Libre (flexible)	Semi-imposee
Tests	Karma + Jasmine integres	Jest (separe)	Vitest (separe)
Adapte HDS	Excellent	Correct	Correct
Soutenu par	Google	Meta	Communaute

1.2 Blocs de Construction Fondamentaux

Une application Angular medicale s'articule autour de ces elements cles :

Bloc	Role dans une App Medicale
NgModules	Regroupent les fonctionnalites : PatientModule, ConsultationModule, OrdonnanceModule
Components	Vues : FichePatient, ListeConsultations, CarteOrdonnance, TableauBord
Templates	HTML etendu avec syntaxe Angular : binding, directives, pipes
Services	Logique metier : PatientService, FHIR_ApiService, AuthService, AuditService
Router	Navigation : /patients/:id, /consultations/new, /ordonnances/:id
Directives	Comportements : surlignage alerte, masquage donnees sensibles
Pipes	Transformations : formatage INS, age a partir date naissance, codes CIM-10
Guards	Securite : verification role (medecin, infirmier, admin), authentication

1.3 Standards Medicaux & Contexte Reglementaire

Notions cles pour le developpement medical en France

- HDS (Hebergement de Donnees de Sante) : certification obligatoire pour stocker des donnees patients en France
- RGPD / GDPR : anonymisation, consentement explicite, droit a l'oubli - a integrer des la conception
- FHIR (Fast Healthcare Interoperability Resources) : standard HL7 pour l'echange de donnees medicales via API REST
- INS (Identite Nationale de Sante) : identifiant unique patient en France (NIR + traits d'identite)
- IHE Profiles : standards d'integration hospitaliere (XDS, PIX, PDQ) pour l'interoperabilite
- SNOMED CT / CIM-10 / LOINC : terminologies medicales standardisees pour le codage
- ISO 27001 / ISO 13485 : normes securite et qualite pour les logiciels medicaux

Module 2 : Installation & Premier Projet Medical

Duree : 25 minutes | Format : TP Pratique

2.1 Prerequis Techniques

- Node.js 18+ LTS — nodejs.org
- npm 9+ (fourni avec Node) ou yarn 1.22+
- VS Code avec les extensions : Angular Language Service, ESLint, Prettier
- Chrome + extension Angular DevTools
- Postman ou Insomnia pour tester les API FHIR

2.2 Installation du CLI Angular

```
# Installation globale du CLI Angular
npm install -g @angular/cli

# Verification
ng version

# Resultat attendu : Angular CLI: 17.x.x   Node: 18.x.x
```

2.3 Creation du Projet SGP (Systeme de Gestion de Patients)

```
# Creation du projet avec routing et SCSS
ng new sgp-medical --routing --style=scss

# Repondre aux questions du CLI :
# ? Enable Server-Side Rendering (SSR)? No

cd sgp-medical

# Lancer le serveur de developpement
ng serve --open
# Application disponible sur http://localhost:4200

# Generer les modules principaux
ng generate module features/patients --routing
ng generate module features/consultations --routing
ng generate module features/ordonnances --routing
ng generate module core
ng generate module shared
```

2.4 Structure du Projet Medical

```
sgp-medical/src/app/
  core/                                <- Services singleton (charges une seule fois)
```

```

services/
  auth.service.ts      <- Authentification & gestion des roles
  audit.service.ts     <- Journalisation des acces (obligation legale)
  notification.service.ts <- Alertes medicales en temps reel
interceptors/
  auth.interceptor.ts  <- Injection token JWT dans chaque requete
  audit.interceptor.ts <- Journalisation automatique des appels API
guards/
  auth.guard.ts        <- Verification de l'authentification
  role.guard.ts        <- Controle d'accès base sur les roles (RBAC)
shared/
  components/
    patient-badge/     <- Badge avec initiales et statut
    alert-banner/      <- Banniere d'alerte medicale
    vitals-chart/      <- Graphique constantes vitales
  pipes/
    age.pipe.ts        <- Calcul age depuis date de naissance
    ins-format.pipe.ts <- Formatage Identite Nationale de Sante
    cim10.pipe.ts      <- Libelle code CIM-10
features/
  patients/
    components/
      patient-list/    <- Liste avec filtres, tri, pagination
      patient-card/    <- Carte recapitulative patient
      patient-form/    <- Formulaire creation/modification
      patient-detail/  <- Dossier patient complet
    services/
      patient.service.ts <- CRUD patients (etat local)
      patient-api.service.ts <- Appels API FHIR Patient
    models/
      patient.model.ts  <- Interface Patient (conforme FHIR R4)
  consultations/
  ordonnances/
app-routing.module.ts
app.module.ts

```

2.5 Commandes CLI de Reference

Commande	Raccourci	Utilite
ng generate component <n>	ng g c <n>	Creer un composant (4 fichiers)
ng generate service <n>	ng g s <n>	Creer un service
ng generate module <n>	ng g m <n>	Creer un NgModule
ng generate pipe <n>	ng g p <n>	Creer un pipe
ng generate guard <n>	ng g g <n>	Creer un garde de route
ng generate directive <n>	ng g d <n>	Creer une directive
ng build -- configuration=production	ng b	Build de production

<code>ng test --code-coverage</code>	<code>ng t</code>	Tests unitaires + couverture
<code>ng lint</code>		Analyse statique ESLint

Module 3 : Composants Medicaux Approfondis

Duree : 40 minutes | Format : Cours + 3 TP

3.1 Le Decorateur @Component

Un composant est la brique fondamentale de l'interface utilisateur. Il s'agit d'une classe TypeScript decoree avec @Component qui fournit les metadonnees necessaires a Angular.

```
// src/app/features/patients/components/patient-card/patient-card.component.ts
import { Component, Input, Output, EventEmitter, OnInit, OnDestroy } from
 '@angular/core';
import { Subject } from 'rxjs';
import { takeUntil } from 'rxjs/operators';
import { Patient } from '../../../models/patient.model';

@Component({
  selector: 'app-patient-card',
  templateUrl: './patient-card.component.html',
  styleUrls: ['./patient-card.component.scss'],
  // changeDetection: ChangeDetectionStrategy.OnPush // Optimisation performances
})
export class PatientCardComponent implements OnInit, OnDestroy {
  @Input() patient!: Patient; // Donnees patient du parent
  @Input() showSensitiveData = false; // Controle affichage donnees sensibles
  @Output() patientSelected = new EventEmitter<Patient>();
  @Output() urgenceSignalee = new EventEmitter<{ patient: Patient; motif: string
}>();

  private destroy$ = new Subject<void>();

  get age(): number {
    const now = new Date();
    const birth = new Date(this.patient.dateNaissance);
    return now.getFullYear() - birth.getFullYear();
  }

  get initialles(): string {
    return `${this.patient.prenom[0]}${this.patient.nom[0]}`.toUpperCase();
  }

  ngOnInit(): void {
    console.log('Fiche patient initialisee :', this.patient.ins);
  }

  signalerUrgence(motif: string): void {
    this.urgenceSignalee.emit({ patient: this.patient, motif });
  }

  ngOnDestroy(): void {
    this.destroy$.next();
  }
}
```



```
    this.destroy$.complete();  
  }  
}
```

3.2 Le Modele Patient (conforme FHIR R4)

```
// src/app/features/patients/models/patient.model.ts  
  
export type Sexe = 'M' | 'F' | 'I' | 'U'; // Male, Female, Indetermine, Unknown  
export type GroupeSanguin = 'A+' | 'A-' | 'B+' | 'B-' | 'AB+' | 'AB-' | 'O+' | 'O-';  
export type StatutPatient = 'actif' | 'inactif' | 'decede' | 'transfere';  
export type NiveauUrgence = 'vert' | 'jaune' | 'orange' | 'rouge';  
  
export interface Adresse {  
  ligne1: string;  
  ligne2?: string;  
  codePostal: string;  
  ville: string;  
  pays: string;  
}  
  
export interface Allergie {  
  substance: string;  
  reaction: string;  
  severite: 'legere' | 'moderee' | 'severe';  
  dateDeclaration: Date;  
}  
  
export interface ConstantesVitales {  
  tensionSystolique?: number; // mmHg  
  tensionDiastolique?: number; // mmHg  
  frequenceCardiaque?: number; // bpm  
  temperature?: number; // Celsius  
  saturationO2?: number; // %  
  poids?: number; // kg  
  taille?: number; // cm  
  dateMesure: Date;  
}  
  
export interface Patient {  
  id: string; // UUID interne  
  ins: string; // Identite Nationale de Sante (NIR)  
  nom: string;  
  prenom: string;  
  dateNaissance: Date;  
  sexe: Sexe;  
  groupeSanguin?: GroupeSanguin;  
  adresse: Adresse;  
  telephone: string;
```

```
email?: string;
medecinTraitantId: string;
statut: StatutPatient;
niveauUrgence?: NiveauUrgence;
allergies: Allergie[];
antecedents: string[];
traitementEnCours: string[];
dernieresConstantes?: ConstantesVitales;
dateCreation: Date;
dateDerniereModification: Date;
}
```

3.3 Communication Entre Composants

@Input - Du Parent vers l'Enfant

```
// TEMPLATE DU PARENT (patient-list.component.html)
<app-patient-card
  *ngFor='let p of patients'
  [patient]='p'
  [showSensitiveData]='currentUser.role === "medecin"'
  (patientSelected)='onPatientSelectionne($event)'
  (urgenceSignalee)='onUrgence($event)'
></app-patient-card>

// COMPOSANT ENFANT (patient-card.component.ts)
@Input() patient!: Patient;
@Input() showSensitiveData = false;

// Angular 16+ - Input obligatoire (erreur a la compilation si absent)
@Input({ required: true }) patient!: Patient;
```

@Output - De l'Enfant vers le Parent

```
// COMPOSANT ENFANT
@Output() patientSelected = new EventEmitter<Patient>();
@Output() urgenceSignalee = new EventEmitter<{ patient: Patient; motif: string }>();

selectionnerPatient(): void {
  this.patientSelected.emit(this.patient);
}

// COMPOSANT PARENT
onPatientSelectionne(patient: Patient): void {
  this.router.navigate(['/patients', patient.id]);
}

onUrgence(data: { patient: Patient; motif: string }): void {
  this.urgenceService.declencherAlerte(data.patient, data.motif);
}
```

```
this.auditService.log('URGENCE', data.patient.id, data.motif);
}
```

Projection de Contenu avec ng-content

```
// dossier-patient.component.html - Composant conteneur
<div class='dossier'>
  <div class='dossier__header'>
    <ng-content select='[slot=entete]'></ng-content>
  </div>
  <div class='dossier__corps'>
    <ng-content></ng-content>
  </div>
  <div class='dossier__pied'>
    <ng-content select='[slot=actions]'></ng-content>
  </div>
</div>

<!-- UTILISATION -->
<app-dossier-patient>
  <app-entete-patient slot='entete' [patient]='patient'></app-entete-patient>
  <app-historique-consultations [patientId]='patient.id'></app-historique-consultations>
  <app-actions-patient slot='actions' [patient]='patient'></app-actions-patient>
</app-dossier-patient>
```

3.4 Cycle de Vie des Composants

Hook	Declenchement	Cas d'Usage Medical
ngOnChanges	Avant ngOnInit, a chaque changement @Input	Recalculer l'IMC quand poids/taille changent
ngOnInit	Après le premier ngOnChanges	Charger le dossier patient, s'abonner aux alertes
ngDoCheck	A chaque cycle de detection	Surveillance personnalisee (constantes critiques)
ngAfterContentInit	Après projection ng-content	Acceder aux composants projetes
ngAfterViewInit	Après init de la vue	Initialiser graphiques (Chart.js, D3)
ngOnDestroy	Avant suppression du composant	Se desabonner, annuler timers d'alerte

```
// patient-monitoring.component.ts - Surveillance en temps reel
export class PatientMonitoringComponent implements OnInit, OnDestroy {
  private alerteTimer?: ReturnType<typeof setInterval>;
  private readonly destroy$ = new Subject<void>();

  ngOnInit(): void {
```

```
// Chargement initial du dossier
this.patientService.getDossier(this.patientId)
  .pipe(takeUntil(this.destroy$))
  .subscribe(dossier => this.dossier = dossier);

// Surveillance constantes vitales toutes les 30 secondes
this.alerteTimer = setInterval(() => {
  this.verifierConstantesCritiques();
}, 30_000);
}

private verifierConstantesCritiques(): void {
  const c = this.dossier?.dernieresConstantes;
  if (!c) return;
  if (c.saturationO2 && c.saturationO2 < 90) {
    this.notificationService.urgence(`SpO2 critique : ${c.saturationO2}%`);
  }
  if (c.tensionSystolique && c.tensionSystolique > 180) {
    this.notificationService.urgence(`HTA severe : ${c.tensionSystolique} mmHg`);
  }
}

ngOnDestroy(): void {
  clearInterval(this.alerteTimer); // IMPERATIF : stopper le timer
  this.destroy$.next();
  this.destroy$.complete();
}
}
```

3.5 TP - Construire la PatientCard

TP 3.5 - Instructions

- 1. ng generate component features/patients/components/patient-card
- 2. Creer les interfaces Patient, Allergie, ConstantesVitales dans patient.model.ts
- 3. Ajouter @Input() patient: Patient et @Input() showSensitiveData: boolean
- 4. Ajouter @Output() patientSelected et @Output() urgenceSignalee
- 5. Afficher : nom/prenom, age calcule, INS (masque si !showSensitiveData), statut
- 6. Afficher un badge de couleur selon le niveauUrgence (vert/jaune/orange/rouge)
- 7. Bouton 'Urgence' qui ouvre une modale et emet urgenceSignalee
- 8. Icone allergie si allergies.length > 0 avec tooltip sur la liste

Module 4 : Templates & Liaison de Donnees

Duree : 30 minutes | Format : Cours + 2 TP

4.1 Les Quatre Types de Liaison de Donnees

Type	Syntaxe	Direction	Exemple Medical
Interpolation	{{ expr }}	TS vers HTML	{{ patient.nom }} {{ patient.prenom }}
Property Binding	[prop]="expr"	TS vers HTML	[disabled]="!patient.consentement"
Event Binding	(event)="fn()"	HTML vers TS	(click)="ajouterAllergie()"
Two-Way Binding	[(ngModel)]="prop"	Bidirectionnel	[(ngModel)]="diagnostic"

4.2 Exemples de Liaisons dans un Contexte Medical

```
<!-- patient-detail.component.html -->

<!-- 1. INTERPOLATION - Affichage de donnees patient -->
<h1>{{ patient.prenom }} {{ patient.nom }}</h1>
<p>Age : {{ patient | age }} ans</p>
<p>INS : {{ patient.ins | insFormat }}</p>
<p>Derniere consultation : {{ patient.derniereConsultation | date:'dd/MM/yyyy' }}</p>
<p>IMC : {{ calculerIMC(patient) | number:'1.1-1' }} kg/m2</p>

<!-- 2. PROPERTY BINDING - Proprietes DOM/composants -->
<img [src]="patient.photoUrl || '/assets/avatar-default.svg'" [alt]="patient.nom">
<button [disabled]="!peutModifier || patient.statut === 'decede'">Modifier</button>
<div [class.alerte-critique]="patient.niveauUrgence === 'rouge'">...</div>
<div [style.border-color]="getCouleurUrgence(patient.niveauUrgence)">...</div>
<app-constantes [data]="patient.dernieresConstantes" [showAlerts]="true">

<!-- 3. EVENT BINDING - Reactions aux actions utilisateur -->
<button (click)="declarerAllergie()">+ Allergie</button>
<button (click)="$event.stopPropagation(); signalerUrgence(patient)">URGENCE</button>
<input (input)="rechercherPatient($event)" (keyup.escape)="reinitialiser()">
<select (change)="changerStatut($event)">

<!-- 4. TWO-WAY BINDING (necessite FormsModule) -->
<input [(ngModel)]="consultation.diagnostic" placeholder='Diagnostic principal'>
<textarea [(ngModel)]="consultation.observations"></textarea>
<select [(ngModel)]="consultation.codesCIM10">
```

4.3 Pipes Medicaux Personnalisés

```
// src/app/shared/pipes/age.pipe.ts
@Pipe({ name: 'age', pure: true })
export class AgePipe implements PipeTransform {
  transform(dateNaissance: Date | string): number {
    const birth = new Date(dateNaissance);
    const today = new Date();
    let age = today.getFullYear() - birth.getFullYear();
    const m = today.getMonth() - birth.getMonth();
    if (m < 0 || (m === 0 && today.getDate() < birth.getDate())) age--;
    return age;
  }
}

// src/app/shared/pipes/ins-format.pipe.ts
// Formate l'INS : 1850612075047 -> 1 85 06 12 075 047 (clé)
@Pipe({ name: 'insFormat' })
export class InsFormatPipe implements PipeTransform {
  transform(ins: string, masquer = false): string {
    if (masquer) return '* * * * *';
    if (!ins || ins.length < 13) return ins;
    return `${ins[0]} ${ins.slice(1,3)} ${ins.slice(3,5)} ${ins.slice(5,7)} ` +
      `${ins.slice(7,10)} ${ins.slice(10,13)} ${ins.slice(13)} `;
  }
}

// src/app/shared/pipes/cim10.pipe.ts
@Pipe({ name: 'cim10' })
export class Cim10Pipe implements PipeTransform {
  private readonly codes: Record<string, string> = {
    'J44': 'Bronchopneumopathie chronique obstructive',
    'I10': 'Hypertension arterielle essentielle',
    'E11': 'Diabete de type 2',
    'F32': 'Episode depressif',
    // ... charger depuis un service en pratique
  };
  transform(code: string): string {
    return this.codes[code] ?? code;
  }
}

<!-- Utilisation dans les templates -->
<span>{{ patient.dateNaissance | age }} ans</span>
<span>{{ patient.ins | insFormat:!medecinConnecte }}</span>
<li *ngFor='let code of consultation.codesCIM10'>{{ code | cim10 }}</li>
<span>{{ tension | number:'1.0-0' }} mmHg</span>
<span>{{ dateConsultation | date:'EEEE dd MMMM yyyy':'fr' }}</span>
```

Module 5 : Directives Integrees & Personnalisees

Duree : 30 minutes | Format : Cours + 3 TP

5.1 Directives Structurelles

*ngIf - Affichage Conditionnel

```
<!-- Afficher les allergies seulement si present -->
<section *ngIf="patient.allergies.length > 0; else aucuneAllergie">
  <h3>Allergies connues ({{ patient.allergies.length }})</h3>
  <app-allergie-card *ngFor='let a of patient.allergies' [allergie]='a'>
  </app-allergie-card>
</section>

<ng-template #aucuneAllergie>
  <p class='info'>Aucune allergie connue declaree</p>
</ng-template>

<!-- Accces conditionnel selon le role -->
<div *ngIf="utilisateur.role === 'medecin' || utilisateur.role === 'admin'">
  <app-ordonnance-editor [patient]='patient'></app-ordonnance-editor>
</div>

<!-- Angular 17+ : nouvelle syntaxe @if -->
@if (patient.niveauUrgence === 'rouge') {
  <div class='alerte-critique'>
    <span>URGENCE VITALE - Code rouge</span>
  </div>
} @else if (patient.niveauUrgence === 'orange') {
  <div class='alerte-urgence'>Patient necessitant une attention rapide</div>
} @else {
  <div class='statut-normal'>Etat stable</div>
}
```

*ngFor - Listes Medicales

```
<!-- Liste des patients avec toutes les variables exportees -->
<div class='liste-patients'>
  <app-patient-card
    *ngFor="let patient of patientsFiltres;
      let i = index;
      let isFirst = first;
      let isLast = last;
      trackBy: trackByPatientId"
    [patient]='patient'
    [class.premier-patient]='isFirst'
    (patientSelected)='ouvrirDossier($event)'>
  </app-patient-card>
</div>

// TOUJOURS utiliser trackBy pour de grandes listes de patients !
```

```

trackByPatientId(index: number, patient: Patient): string {
  return patient.id; // Identifiant unique UUID
}

<!-- Liste des traitements en cours -->
<ul class='traitements'>
  <li *ngFor='let traitement of patient.traitements; let impair = odd'
    [class.impair]='impair'>
    <span class='medicament'>{{ traitement }}</span>
  </li>
</ul>

<!-- Angular 17+ @for avec @empty -->
@for (consultation of consultations; track consultation.id; let n = $count) {
  <app-consultation-item [consultation]='consultation' />
} @empty {
  <p class='vide'>Aucune consultation enregistree</p>
}

```

*ngSwitch - Statuts et Niveaux d'Urgence

```

<!-- Badge de niveau d'urgence (triage CCMU) -->
<div [ngSwitch]='patient.niveauUrgence'>
  <span *ngSwitchCase="'vert'" class='badge-vert'>CCMU 1 - Stable</span>
  <span *ngSwitchCase="'jaune'" class='badge-jaune'>CCMU 2 - Peu grave</span>
  <span *ngSwitchCase="'orange'" class='badge-orange'>CCMU 3 - Grave</span>
  <span *ngSwitchCase="'rouge'" class='badge-rouge'>CCMU 4/5 - URGENT</span>
  <span *ngSwitchDefault class='badge-gris'>Non evalue</span>
</div>

<!-- Icone groupe sanguin -->
<span [ngSwitch]='patient.groupeSanguin'>
  <ng-container *ngSwitchCase="'O-'>Donneur universel</ng-container>
  <ng-container *ngSwitchCase="'AB+'>Receveur universel</ng-container>
  <ng-container *ngSwitchDefault>{{ patient.groupeSanguin }}</ng-container>
</span>

```

5.2 Directives d'Attribut

NgClass & NgStyle dans un Contexte Medical

```

<!-- Coloration dynamique selon les constantes vitales -->
<td [ngClass]="{
  'valeur-normale': isNormal(constante.saturationO2, 95, 100),
  'valeur-attention': isBetween(constante.saturationO2, 90, 94),
  'valeur-critique': constante.saturationO2 < 90,
  'valeur-inconnue': !constante.saturationO2
}">
  {{ constante.saturationO2 ?? 'N/A' }} %
</td>

```



```
<!-- Style dynamique pour graphique de tension -->
<div class='barre-tension' [ngStyle]="{
  'width': (constante.tensionSystolique / 200 * 100) + '%',
  'background-color': constante.tensionSystolique > 140 ? '#e74c3c' : '#27ae60'
}"></div>
```

Directive Personnalisée : Masquage de Donnees Sensibles

```
// ng generate directive directives/sensitive-data
// src/app/shared/directives/sensitive-data.directive.ts

@Directive({ selector: '[appSensitiveData]' })
export class SensitiveDataDirective implements OnInit {
  @Input() appSensitiveData = true; // true = masquer par default
  @Input() maskChar = '*';

  private originalText = '';
  private isVisible = false;

  constructor(private el: ElementRef, private renderer: Renderer2,
    private authService: AuthService) {}

  ngOnInit(): void {
    if (this.appSensitiveData && !this.authService.peutVoirDonneesSensibles()) {
      this.originalText = this.el.nativeElement.textContent;
      this.masquer();
    }
  }

  @HostListener('click')
  toggleVisibilite(): void {
    if (!this.authService.peutVoirDonneesSensibles()) return;
    this.isVisible ? this.masquer() : this.afficher();
    this.isVisible = !this.isVisible;
    this.auditService.log('VIEW_SENSITIVE',
      this.el.nativeElement.dataset['patientId']);
  }

  private masquer(): void {
    const masked = this.maskChar.repeat(this.originalText.length);
    this.renderer.setProperty(this.el.nativeElement, 'textContent', masked);
    this.renderer.addClass(this.el.nativeElement, 'donnee-masquee');
  }

  private afficher(): void {
    this.renderer.setProperty(this.el.nativeElement, 'textContent',
      this.originalText);
    this.renderer.removeClass(this.el.nativeElement, 'donnee-masquee');
  }
}
```

```
<!-- Utilisation -->  
<span [appSensitiveData]='true' [attr.data-patient-id]='patient.id'>  
  {{ patient.ins }}  
</span>  
<span [appSensitiveData]='!currentUser.isMedecin'>  
  {{ patient.email }}  
</span>
```

Module 6 : Services & Injection de Dependances

Duree : 25 minutes | Format : Cours + 2 TP

6.1 Services Medicaux Essentiels

Quand creer un service ?

- Partager des donnees entre plusieurs composants (liste patients, consultations en cours)
- Encapsuler les appels API (FHIR, HL7, services HDS)
- Implementer la logique metier medicale (calcul IMC, scores cliniques, alertes)
- Gerer l'etat global de l'application (patient courant, utilisateur connecte)
- Journaliser les acces aux donnees sensibles (obligation legale - RGPD/HDS)
- Centraliser la gestion des erreurs et les notifications

6.2 PatientService - Gestion d'Etat Local

```
// src/app/features/patients/services/patient.service.ts
import { Injectable } from '@angular/core';
import { BehaviorSubject, Observable, combineLatest } from 'rxjs';
import { map, distinctUntilChanged } from 'rxjs/operators';
import { Patient, NiveauUrgence } from '../models/patient.model';

@Injectable({ providedIn: 'root' })
export class PatientService {
  // Etat interne (mutable, prive)
  private patientsSubject = new BehaviorSubject<Patient[]>([]);
  private patientCourantSubject = new BehaviorSubject<Patient | null>(null);
  private chargementSubject = new BehaviorSubject<boolean>(false);

  // Observables publics (lecture seule)
  readonly patients$ = this.patientsSubject.asObservable();
  readonly patientCourant$ = this.patientCourantSubject.asObservable();
  readonly enChargement$ = this.chargementSubject.asObservable();

  // Flux derives avec operateurs RxJS
  readonly patientsUrgents$ = this.patients$.pipe(
    map(pts => pts.filter(p => p.niveauUrgence === 'rouge' || p.niveauUrgence === 'orange')),
    distinctUntilChanged()
  );

  readonly statistiques$ = this.patients$.pipe(
    map(pts => ({
      total: pts.length,
      actifs: pts.filter(p => p.statut === 'actif').length,
      urgencesRouge: pts.filter(p => p.niveauUrgence === 'rouge').length,
      allergies: pts.filter(p => p.allergies.length > 0).length,
    })))
  );
```

```
    )))
  );

  getPatients(): Patient[] {
    return [...this.patientsSubject.getValue()];
  }

  getPatientById(id: string): Patient | undefined {
    return this.patientsSubject.getValue().find(p => p.id === id);
  }

  definirPatientCourant(patient: Patient | null): void {
    this.patientCourantSubject.next(patient);
  }

  ajouterPatient(patient: Omit<Patient, 'id' | 'dateCreation' | 'dateDerniereModification'>): void {
    const nouveau: Patient = {
      ...patient,
      id: crypto.randomUUID(),
      dateCreation: new Date(),
      dateDerniereModification: new Date()
    };
    const current = this.patientsSubject.getValue();
    this.patientsSubject.next([...current, nouveau]);
  }

  mettreAJourPatient(updated: Patient): void {
    const patients = this.patientsSubject.getValue().map(
      p => p.id === updated.id ? { ...updated, dateDerniereModification: new Date()
    } : p
    );
    this.patientsSubject.next(patients);
    if (this.patientCourantSubject.getValue()?.id === updated.id) {
      this.patientCourantSubject.next(updated);
    }
  }

  modifierNiveauUrgence(id: string, niveau: NiveauUrgence): void {
    const patient = this.getPatientById(id);
    if (patient) this.mettreAJourPatient({ ...patient, niveauUrgence: niveau });
  }
}
```

6.3 AuditService - Journalisation Obligatoire

```
// src/app/core/services/audit.service.ts
// Obligatoire pour la conformite HDS/RGPD

@Injectable({ providedIn: 'root' })
```

```
export class AuditService {
  private readonly auditApiUrl = '/api/audit';

  constructor(
    private http: HttpClient,
    private authService: AuthService
  ) {}

  log(action: AuditAction, patientId?: string, details?: string): void {
    const entree: AuditLog = {
      timestamp: new Date().toISOString(),
      utilisateurId: this.authService.getUtilisateurConnecte()?.id ?? 'inconnu',
      utilisateurRole: this.authService.getRole(),
      action,
      patientId,
      details,
      adresseIP: 'client-side', // L'IP réelle est enregistrée côté serveur
    };
    // Envoi non bloquant - ne pas bloquer l'UI pour les logs
    this.http.post(this.auditApiUrl, entree).subscribe({
      error: err => console.error('Erreur journalisation audit :', err)
    });
  }
}

export type AuditAction =
  | 'CONSULTATION_DOSSIER'
  | 'MODIFICATION_PATIENT'
  | 'CREATION_ORDONNANCE'
  | 'VIEW_SENSITIVE'
  | 'EXPORT_DONNEES'
  | 'SUPPRESSION_DONNEES';
```

Module 7 : Routage Angular Medical

Duree : 25 minutes | Format : Cours + TP

7.1 Configuration des Routes

```
// src/app/app-routing.module.ts
const routes: Routes = [
  // Redirection racine
  { path: '', redirectTo: '/tableau-de-bord', pathMatch: 'full' },

  // Routes principales
  { path: 'tableau-de-bord', component: TableauDeBordComponent },

  // Module Patients (charge en mode eager pour acces rapide)
  { path: 'patients', component: PatientListComponent },
  { path: 'patients/nouveau', component: PatientFormComponent },
  { path: 'patients/:id', component: PatientDetailComponent },
  { path: 'patients/:id/modifier', component: PatientFormComponent },
  { path: 'patients/:id/dossier', component: DossierPatientComponent },

  // Modules charges de maniere paresseuse (lazy loading)
  { path: 'consultations',
    loadChildren: () => import('./features/consultations/consultations.module')
      .then(m => m.ConsultationsModule) },

  { path: 'ordonnances',
    loadChildren: () => import('./features/ordonnances/ordonnances.module')
      .then(m => m.OrdonnancesModule) },

  // Routes protegees par les gardes
  { path: 'admin',
    loadChildren: () => import('./features/admin/admin.module').then(m =>
m.AdminModule),
    canActivate: [authGuard],
    canActivateChild: [roleGuard],
    data: { rolesRequis: ['admin'] } },

  // Page de connexion (accessible sans authentication)
  { path: 'connexion', component: ConnexionComponent,
    canActivate: [dejaCnxGuard] }, // Redirige si deja connecte

  // Wildcard - doit etre la derniere route
  { path: '**', component: PageIntrouvableComponent }
];
```

7.2 Navigation dans les Templates

```
<!-- navigation.component.html -->
```

```
<nav class='nav-medical'>
  <a routerLink='/tableau-de-bord' routerLinkActive='actif'
    [routerLinkActiveOptions]="{exact: true}">
    Tableau de bord
  </a>
  <a routerLink='/patients' routerLinkActive='actif'>Patients</a>
  <a routerLink='/consultations' routerLinkActive='actif'>Consultations</a>

  <!-- Lien dynamique vers le dossier d'un patient -->
  <a [routerLink]="['/patients', patient.id, 'dossier']">
    Ouvrir le dossier
  </a>

  <!-- Lien avec parametres de requete -->
  <a [routerLink]="['/patients']"
    [queryParams]="{urgence: 'rouge', tri: 'nom', page: 1}">
    Patients urgents
  </a>
</nav>

<!-- Le composant route s'affiche ici -->
<main class='contenu-principal'>
  <router-outlet></router-outlet>
</main>
```

7.3 Lecture des Parametres de Route

```
// patient-detail.component.ts
@Component({ ... })
export class PatientDetailComponent implements OnInit {
  patient?: Patient;
  ongletActif = 'dossier';

  constructor(
    private route: ActivatedRoute,
    private router: Router,
    private patientService: PatientService,
    private auditService: AuditService
  ) {}

  ngOnInit(): void {
    // Lecture du parametre :id
    this.route.paramMap.subscribe(params => {
      const id = params.get('id') ?? '';
      this.patient = this.patientService.getPatientById(id);

      if (!this.patient) {
        this.router.navigate(['/patients']);
        return;
      }
    })
  }
}
```

```
// Journalisation obligatoire de chaque consultation du dossier
this.auditService.log('CONSULTATION_DOSSIER', id);
this.patientService.definirPatientCourant(this.patient);
});

// Lecture des query params : /patients/:id?onglet=constantes
this.route.queryParamMap.subscribe(q => {
  this.ongletActif = q.get('onglet') ?? 'dossier';
});
}

changerOnglet(onglet: string): void {
  this.router.navigate([], {
    relativeTo: this.route,
    queryParams: { onglet },
    queryParamsHandling: 'merge' // Conserver les autres query params
  });
}
}
```

7.4 Gardes de Route Medicaux

```
// src/app/core/guards/role.guard.ts
// Controle d'Acces Base sur les Roles (RBAC)

export const roleGuard: CanActivateFn = (route, state) => {
  const auth = inject(AuthService);
  const router = inject(Router);

  const rolesRequis: string[] = route.data?.['rolesRequis'] ?? [];
  const roleUtilisateur = auth.getRole();

  if (rolesRequis.length === 0 || rolesRequis.includes(roleUtilisateur)) {
    return true;
  }

  // Acces refuse - journaliser la tentative
  inject(AuditService).log('ACCES_REFUSE', undefined,
    `Route: ${state.url}, Role: ${roleUtilisateur}, Requis:
    ${rolesRequis.join(',')}`
  );

  return router.createUrlTree(['/acces-refuse']);
};

// CanDeactivate - Proteger les formulaires non sauvegardes
export const formulaireNonSauvegardeGuard: CanDeactivateFn<PatientFormComponent> =
  (component) => {
    if (component.formulaire.dirty) {
```



```
    return confirm(  
      'Des modifications non sauvegardees seront perdues. Continuer quand meme ?'  
    );  
  }  
  return true;  
};
```

Module 8 : Formulaires Reactifs Medicaux

Duree : 20 minutes | Format : Cours + TP

8.1 Formulaire de Saisie Patient

```
    groupeSanguin: [''],
    medecinTraitantId: ['', Validators.required],
    antecedents: this.fb.array([]),
    allergies: this.fb.array([]),
  });

  if (patientExistant) {
    this.patientForm.patchValue(patientExistant);
    patientExistant.antecedents.forEach(a => this.ajouterAntecedent(a));
  }
}

// FormArrays
get antecedents(): FormArray { return this.patientForm.get('antecedents') as FormArray; }
get allergies(): FormArray { return this.patientForm.get('allergies') as FormArray; }

ajouterAntecedent(valeur = ''): void {
  this.antecedents.push(this.fb.control(valeur, Validators.required));
}

ajouterAllergie(): void {
  this.allergies.push(this.fb.group({
    substance: ['', Validators.required],
    reaction: ['', Validators.required],
    severite: ['moderee', Validators.required],
    dateDeclaration: [new Date(), Validators.required],
  }));
}

// Valideur personnelisé : date dans le passé
valideurDatePassee(ctrl: AbstractControl) {
  return new Date(ctrl.value) < new Date() ? null : { dateAvenir: true };
}

onSoumettre(): void {
  if (this.patientForm.valid) {
    if (this.modeEdition) {
      this.patientService.mettreAJourPatient(this.patientForm.value);
    } else {
      this.patientService.ajouterPatient(this.patientForm.value);
    }
  } else {
    this.patientForm.markAllAsTouched();
  }
}
}
```

```
<!-- patient-form.component.html (extrait) -->
<form [formGroup]='patientForm' (ngSubmit)='onSoumettre()' novalidate>
```

```
<h2>{{ modeEdition ? 'Modifier le patient' : 'Nouveau patient' }}</h2>

<!-- Champ INS avec validation -->
<div class='champ'>
  <label>INS (Identite Nationale de Sante) *</label>
  <input formControlName='ins' type='text' placeholder='1850612075047XX'>
  <div *ngIf="patientForm.get('ins')?.invalid &&
patientForm.get('ins')?.touched">
    <span *ngIf="patientForm.get('ins')?.errors?.['required']">L'INS est
obligatoire</span>
    <span *ngIf="patientForm.get('ins')?.errors?.['pattern']">
      Format invalide (13 chiffres, commençant par 1 ou 2)
    </span>
  </div>
</div>

<!-- Groupe adresse -->
<fieldset formGroupName='adresse'>
  <legend>Adresse</legend>
  <input formControlName='lign1' placeholder='Numero et nom de rue'>
  <input formControlName='codePostal' placeholder='75001'>
  <input formControlName='ville' placeholder='Paris'>
</fieldset>

<!-- FormArray antecedents -->
<div formArrayName='antecedents'>
  <label>Antecedents medicaux</label>
  <div *ngFor='let atcd of antecedents.controls; let i = index' class='rangee'>
    <input [formControlName]='i' placeholder='Ex: Diabete type 2, HTA'>
    <button type='button' (click)='antecedents.removeAt(i)'>Supprimer</button>
  </div>
</div>
<button type='button' (click)='ajouterAntecedent()'>+ Antecedent</button>

<div class='actions'>
  <button type='submit' [disabled]='patientForm.invalid'>
    {{ modeEdition ? 'Enregistrer les modifications' : 'Creer le patient' }}
  </button>
  <button type='button' routerLink='/patients'>Annuler</button>
</div>
</form>
```

Module 9 : HTTP Client, FHIR & Securite

Duree : 15 minutes | Format : Cours + TP

9.1 Introduction a FHIR (Fast Healthcare Interoperability Resources)

FHIR R4 est le standard HL7 pour l'echange de donnees medicales via des API REST. En France, le programme Mon Espace Sante et les ANS API utilisent FHIR R4.

Ressource FHIR	URL API	Description
Patient	GET /Patient/:id	Dossier patient (identite, contact)
Observation	GET /Observation?patient=:id	Constantes vitales, resultats labo
MedicationRequest	GET /MedicationRequest?patient=:id	Ordonnances et prescriptions
Appointment	GET /Appointment?patient=:id	Rendez-vous planifies
AllergyIntolerance	GET /AllergyIntolerance?patient=:id	Allergies et intolerances
Condition	GET /Condition?patient=:id	Diagnostics et pathologies (CIM-10)
DiagnosticReport	GET /DiagnosticReport?patient=:id	Comptes rendus biologiques/radio

9.2 Service API FHIR

```
// src/app/features/patients/services/patient-api.service.ts
import { HttpClient, HttpParams, HttpHeaders } from '@angular/common/http';
import { Observable, throwError } from 'rxjs';
import { map, catchError, retry } from 'rxjs/operators';

@Injectable({ providedIn: 'root' })
export class PatientApiService {
  private readonly fhirBaseUrl = 'https://api.fhir.sante.gouv.fr/r4';

  constructor(private http: HttpClient) {}

  // GET - Recherche patients (parametres de recherche FHIR)
  rechercherPatients(params: {
    nom?: string;
    prenom?: string;
    dateNaissance?: string;
    _count?: number;
  }): Observable<Patient[]> {
    let httpParams = new HttpParams();
    if (params.nom) httpParams = httpParams.set('family', params.nom);
    if (params.prenom) httpParams = httpParams.set('given', params.prenom);
```

```
    if (params.dateNaissance) httpParams = httpParams.set('birthdate',
params.dateNaissance);
    httpParams = httpParams.set('_count', params._count ?? 20);

    return this.http.get<FhirBundle<FhirPatient>>(`${this.fhirBaseUrl}/Patient`, {
      params: httpParams,
      headers: { Accept: 'application/fhir+json' }
    }).pipe(
      map(bundle => bundle.entry?.map(e => this.fhirVersPatient(e.resource)) ??
[]),
      retry(2),
      catchError(this.gererErreur)
    );
  }

  // GET - Dossier patient complet
  getDossierPatient(id: string): Observable<Patient> {
    return this.http.get<FhirPatient>(`${this.fhirBaseUrl}/Patient/${id}`).pipe(
      map(fhir => this.fhirVersPatient(fhir)),
      catchError(this.gererErreur)
    );
  }

  // GET - Observations (constantes vitales)
  getConstantesVitales(patientId: string): Observable<ConstantesVitales[]> {
    const params = new HttpParams()
      .set('patient', patientId)
      .set('category', 'vital-signs')
      .set('_sort', '-date')
      .set('_count', '10');
    return this.http.get<FhirBundle<FhirObservation>>(
      `${this.fhirBaseUrl}/Observation`, { params }
    ).pipe(map(b => b.entry?.map(e => this.fhirVersConstantes(e.resource)) ?? []));
  }

  // POST - Creer un patient
  creerPatient(patient: Patient): Observable<Patient> {
    return this.http.post<FhirPatient>(
      `${this.fhirBaseUrl}/Patient`,
      this.patientVersFhir(patient),
      { headers: { 'Content-Type': 'application/fhir+json' } }
    ).pipe(map(fhir => this.fhirVersPatient(fhir)));
  }

  private gererErreur(error: any): Observable<never> {
    let message = 'Erreur serveur inattendue';
    if (error.status === 0) message = 'Connexion impossible au serveur FHIR';
    if (error.status === 401) message = 'Session expiree - reconnectez-vous';
    if (error.status === 403) message = 'Acces refuse - droits insuffisants';
    if (error.status === 404) message = 'Ressource introuvable';
    console.error('[PatientApiService]', error);
    return throwError(() => new Error(message));
  }
}
```

```
}

private fhirVersPatient(fhir: FhirPatient): Patient {
  return {
    id: fhir.id,
    ins: fhir.identifier?.find(i => i.system ===
'urn:oid:1.2.250.1.213.1.4.8')?.value ?? '',
    nom: fhir.name?.[0]?.family ?? '',
    prenom: fhir.name?.[0]?.given?.[0] ?? '',
    dateNaissance: new Date(fhir.birthDate),
    sexe: fhir.gender === 'male' ? 'M' : 'F',
    // ... mapping complet
  } as Patient;
}
}
```

9.3 Intercepteur d'Authentification

```
// src/app/core/interceptors/auth.interceptor.ts
import { HttpInterceptorFn, HttpRequest, HttpHandlerFn } from
 '@angular/common/http';
import { inject } from '@angular/core';
import { catchError } from 'rxjs/operators';
import { throwError } from 'rxjs';

export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const auth = inject(AuthService);
  const router = inject(Router);
  const audit = inject(AuditService);

  const token = auth.getToken();

  // Injecter le token JWT dans toutes les requetes
  const reqAvecToken = token
    ? req.clone({
        setHeaders: {
          Authorization: `Bearer ${token}`,
          'X-Utilisateur-Id': auth.getUtilisateurConnecte()?.id ?? '',
        }
      })
    : req;

  return next(reqAvecToken).pipe(
    catchError(erreur => {
      if (erreur.status === 401) {
        auth.deconnecter();
        router.navigate(['/connexion']);
      }
      if (erreur.status === 403) {
        audit.log('ACCES_REFUSE', undefined, req.url);
        router.navigate(['/accus-refuse']);
      }
    })
  );
}
```

```
    }  
    return throwError(() => erreur);  
  })  
);  
};
```


Module 10 : Bonnes Pratiques & Prochaines Etapes

Duree : 10 minutes | Format : Discussion & Ressources

10.1 Bonnes Pratiques Angular Medical

Categorie	Bonne Pratique
Securite	Ne jamais stocker de donnees patients en localStorage ou sessionStorage
Securite	Chiffrer toutes les communications (HTTPS/TLS 1.3 minimum)
Securite	Implanter un mecanisme de timeout de session (15 min d'inactivite)
RGPD	Anonymiser les donnees dans les logs, les URL et les messages d'erreur
RGPD	Implanter le consentement explicite avant tout traitement de donnee de sante
Performances	Utiliser OnPush change detection sur les composants recevant des @Input
Performances	Toujours utiliser trackBy avec *ngFor sur les listes de patients
Performances	Lazy-loader les modules (consultations, ordonnances, rapports)
RxJS	Utiliser le pipe async en template au lieu de subscribe/unsubscribe manuel
RxJS	Utiliser takeUntilDestroyed() (Angular 16+) pour les abonnements
Architecture	Separer la logique metier dans les services - garder les composants legers
Audit	Journaliser TOUS les acces aux dossiers patients (obligation HDS)

10.2 Structure Finale du Projet SGP

```
sgp-medical/src/app/  
  core/  
    services/ auth, audit, notification, session-timeout  
    interceptors/ auth, audit, error-handling  
    guards/ auth, role, dejaConnecte, formulaireNonSauvegarde  
  shared/  
    components/ patient-badge, alert-banner, vitals-chart, spinner  
    pipes/ age, ins-format, cim10, time-ago  
    directives/ sensitive-data, role-visibility  
    models/ patient, consultation, ordonnance, utilisateur  
  features/  
    patients/ list, card, form, detail, dossier  
    consultations/ list, detail, form  
    ordonnances/ list, detail, editor  
    tableau-bord/ statistiques, alertes, planning  
    admin/ utilisateurs, roles, audit-logs  
  app-routing.module.ts  
  app.module.ts
```

10.3 Feuille de Route : Pour Aller Plus Loin

- RxJS Avance - operators : switchMap, combineLatest, debounceTime pour la recherche patient
- Angular Signals (v16+) - reactivity fine-grained pour les constantes vitales en temps reel
- NgRx - gestion d'etat centralisee pour les grandes plateformes de sante
- Angular Material + CDK - composants accessibles (WCAG 2.1 AA exige en sante)
- Server-Side Rendering (SSR) - performance et referencement
- Tests Unitaires & E2E - Jasmine/Karma + Cypress pour les parcours critiques
- WebSockets / SSE - surveillance temps reel des constantes vitales
- Micro-Frontends - architectures multi-equipes (module federation)
- Certification HDS - exigences techniques de l'ANSSI pour l'hebergement

10.4 Ressources & Documentation

Ressource	URL / Description
Documentation Angular	angular.dev - documentation officielle (v17+)
HL7 FHIR R4	hl7.org/fhir/R4 - specification officielle FHIR
ANS - Sante.fr	esante.gouv.fr - API et referentiels du numerique en sante FR
Mon Espace Sante API	api.esante.gouv.fr - API nationales (DMP, ordonnances)
CNIL Guide Dev	cnil.fr/guides - guide developpeur RGPD / securite
Angular Material	material.angular.io - composants accessibles
RxJS Documentation	rxjs.dev - reference complete des operateurs
Angular DevTools	Chrome Web Store - debogage et profiling Angular
StackBlitz	stackblitz.com - bac a sable Angular en ligne

Felicitations ! Vous avez termine Angular 101 Medical

Continuez a apprendre, a construire, et a ameliorer les soins par le numerique.